Class BasicObject is a very, very minimal object that is used in certain kinds of metaprogramming, acting as a blank canvas. What's its parent?

```
BasicObject.superclass # => nil
```

So, we've finally reached the end. BasicObject is the root class of our hierarchy of classes. Given any class in any Ruby application, you can ask for its superclass, then the superclass of that class, and so on, and you'll eventually get back to BasicObject.

We've seen that if you call a method in an instance of class Child and that method isn't in Child's class definition, Ruby will look in the parent class. It goes deeper than that, because if the method isn't defined in the parent class, Ruby continues looking in the parent's parent, the parent's parent's parent, and so on, through the ancestors until it runs out of classes. Method lookup in Ruby is actually a little bit more complex, we'll talk more about it in Method Lookup, on page 113.

And this explains our original question about to_s. We can work out why to_s is available in just about every Ruby object. to_s, it turns out, is defined in class Object. Because Object is an ancestor of every Ruby class except BasicObject, instances of every Ruby class have a to_s method defined:

```
class Person
  def initialize(name)
    @name = name
  end
end

p = Person.new("Michael")
puts p
```

produces:

```
#<Person:0x00000001007868c8>
```

We saw in the previous chapter that we can override the to_s method:

```
class Person
  def initialize(name)
    @name = name
  end

  def to_s
    "Person named #{@name}"
  end
end

p = Person.new("Michael")
puts p
```

produces:

```
Person named Michael
```

Armed with our knowledge of subclassing, we now know there's nothing special about this code. The puts method calls to_s on its arguments. In this case, the argument is a Person object. Because class Person defines a to_s method, that method is called. If it doesn't define a to_s method, then Ruby looks for (and finds) to_s in Person's parent class, Object.